



Universidad Nacional Autónoma de México.



Facultad de Ingeniería.

Sistemas Distribuidos.

Profesor: Ing. Óscar Manuel Ruíz Hurtado.

Semestre 2027-1

Laboratorio S5.

Mexi Banco con Docker Compose: lo de la clase, en tu máquina.

Grupo: 2

Alumna:

Valencia Lerdo Dalia Jimena

Paso 1. Clonación del repositorio.

Cloné el repositorio con `git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git`. Git lanzó un aviso de que quedé en detached HEAD, es decir, sobre la versión fija y no sobre una rama.

Imagen 1. Clonación de repositorio.

```
Símbolo del sistema

C:\Users\Jimena\Documents>git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 405.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
warning: refs/tags/v05.1 cd46b8b91a09070f8262c3b8acfb17caf2405cda is not a commit!
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

C:\Users\Jimena\Documents>
```

Paso 2. Levantamiento con Compose.

Con docker el comando `compose up --build -d` se construyó la imagen y se levantaron dos contenedores: db (PostgreSQL 17) y app (la aplicación Java). docker compose ps los muestra como healthy. Los healthchecks confirman que la base acepta conexiones y que la app responde. La app publica el puerto 8080.

Imagen 2. Levantamiento con Compose.

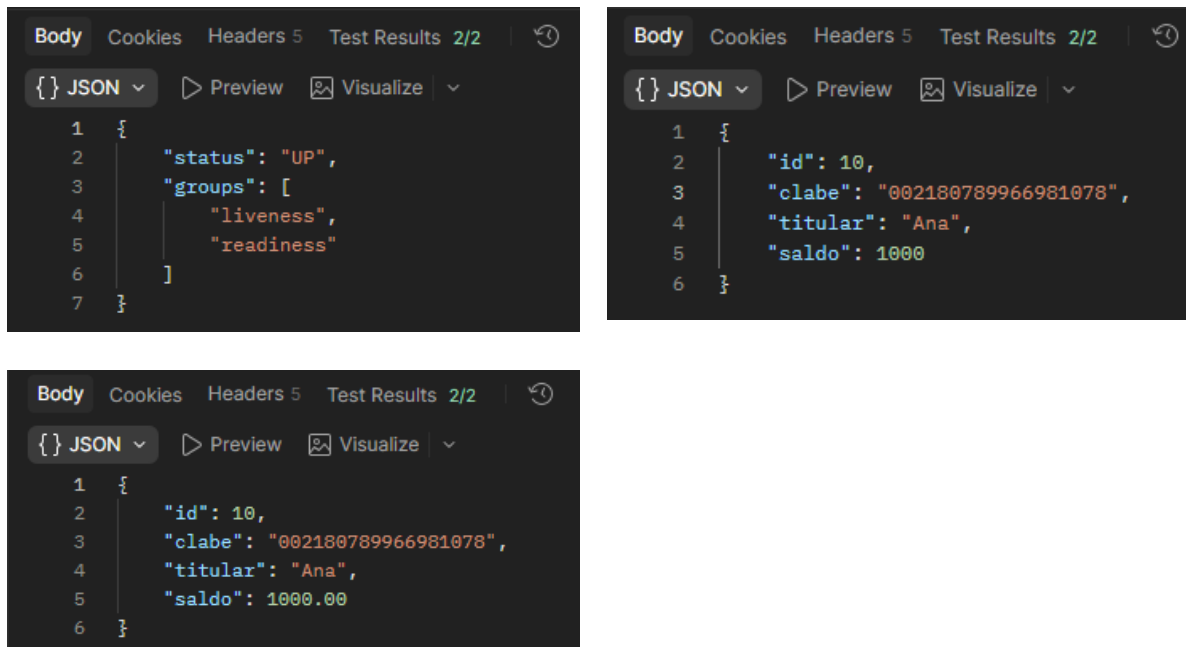
```
Símbolo del sistema

#17 [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar
#17 CACHED
#18 exporting to image
#18 exporting layers done
#18 exporting manifest sha256:085070fac7a0e13a64d8c97c30fbae4dcf310215902f5a57f5138e264803388 done
#18 exporting config sha256:2259ef830b154e31c4482b5932828c92fcc27290714022723a005f8a48977b done
#18 exporting attestation manifest sha256:b5d8977c6581038a3703af072585a40753aea909057eff68f4cf5253eba54cde 0.0s done
#18 exporting manifest list sha256:1376345948066b1aa1e6001af2bce2faaf3de14112bfb967a03050535538f 0.0s done
#18 naming to docker.io/library/mexi-banco-app:latest
#18 naming to docker.io/library/mexi-banco-app:latest done
#18 unpacking to docker.io/library/mexi-banco-app:latest 0.0s done
#18 DONE 0.2s
#19 resolving provenance for metadata file
#19 DONE 0.0s
#19 up 3/3
 0.5s Image mexi-banco-app Built 0.0s Container mexi-banco-db-1 Healthy 1.7s Container mexi-banco-app-1 Running
6.5s
C:\Users\Jimena\Documents\mexi-banco>docker compose ps
NAME                PORTS                IMAGE                COMMAND                SERVICE    CREATED    STATUS
mexi-banco-app-1    sha256:11059c35ea21c4f1a2c6345498dc89540ed62767d255ce48c63bb3fe8d96d42d    "java -jar app.jar"    app                10 hours ago    Up 3 minutes (health)
mexi-banco-db-1     0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp    "docker-entrypoint.s"    db                About a minute ago    Up About a minute (healthy)
mexi-banco-db-1     5432/tcp
```

Paso 3. Pruebas en Postman.

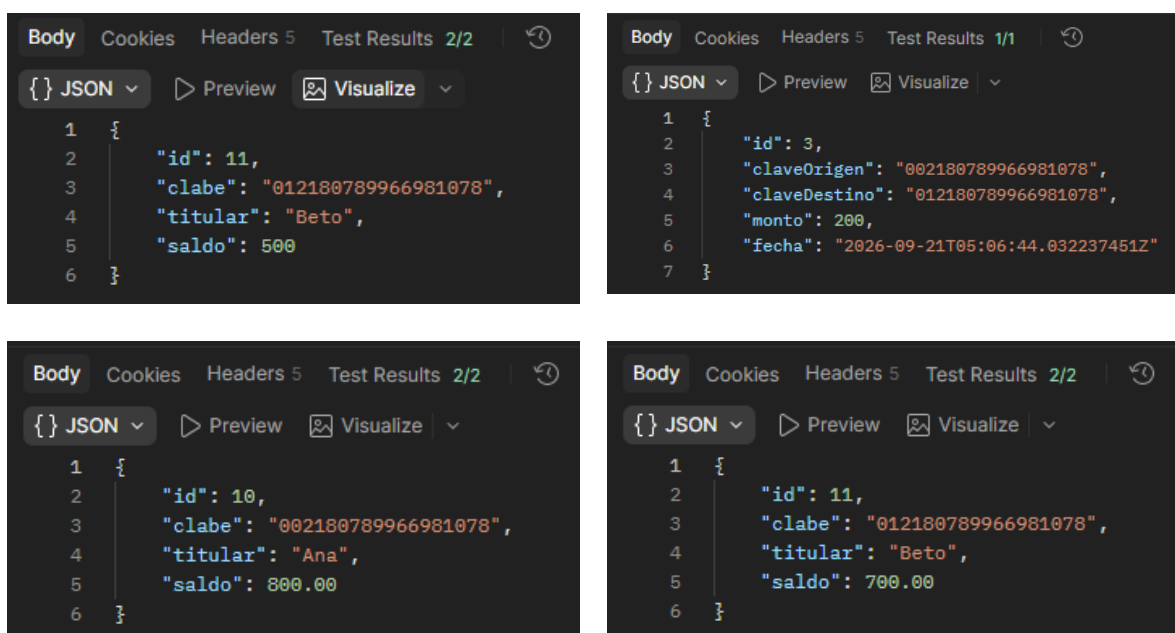
El endpoint de salud respondió UP con los grupos liveness y readiness. Abrí la cuenta de Ana (id 10, saldo 1000) y la consulté.

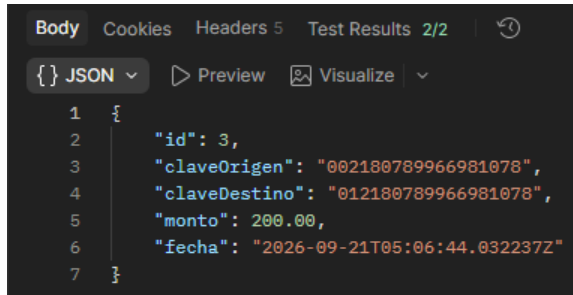
Imagen 3. Arriba y respondiendo.



Abrí la cuenta de Beto con un saldo inicial de \$500 y transferí \$200 de la cuenta de Ana a la de Beto. Ana se quedó en \$800 y Beto en \$700. El dinero se movió completo de una cuenta a otra.

Imagen 4. Transferencia interna.

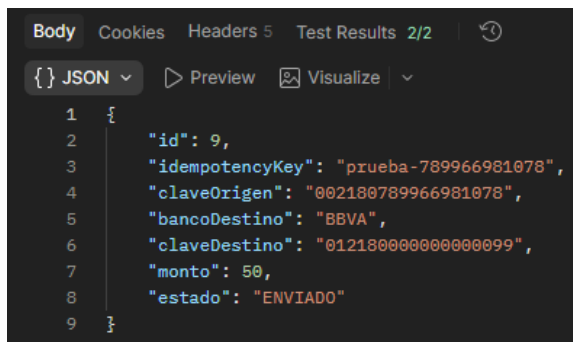




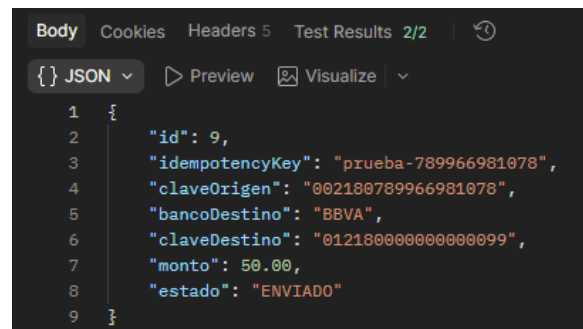
```
1 {
2   "id": 3,
3   "claveOrigen": "002180789966981078",
4   "claveDestino": "012180789966981078",
5   "monto": 200.00,
6   "fecha": "2026-09-21T05:06:44.032237Z"
7 }
```

Mandé el mismo SPEI de \$50 dos veces con la misma Idempotency-Key. Las dos respuestas son idénticas (mismo id 9, estado ENVIADO) y el saldo de Ana bajó solo una vez, de \$800 a \$750. Con esto se demostró que un reintento por una falla de red no realiza el cobro dos veces.

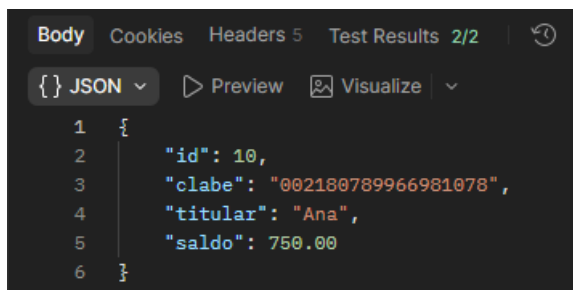
Imagen 5. Idempotencia.



```
1 {
2   "id": 9,
3   "idempotencyKey": "prueba-789966981078",
4   "claveOrigen": "002180789966981078",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50,
8   "estado": "ENVIADO"
9 }
```



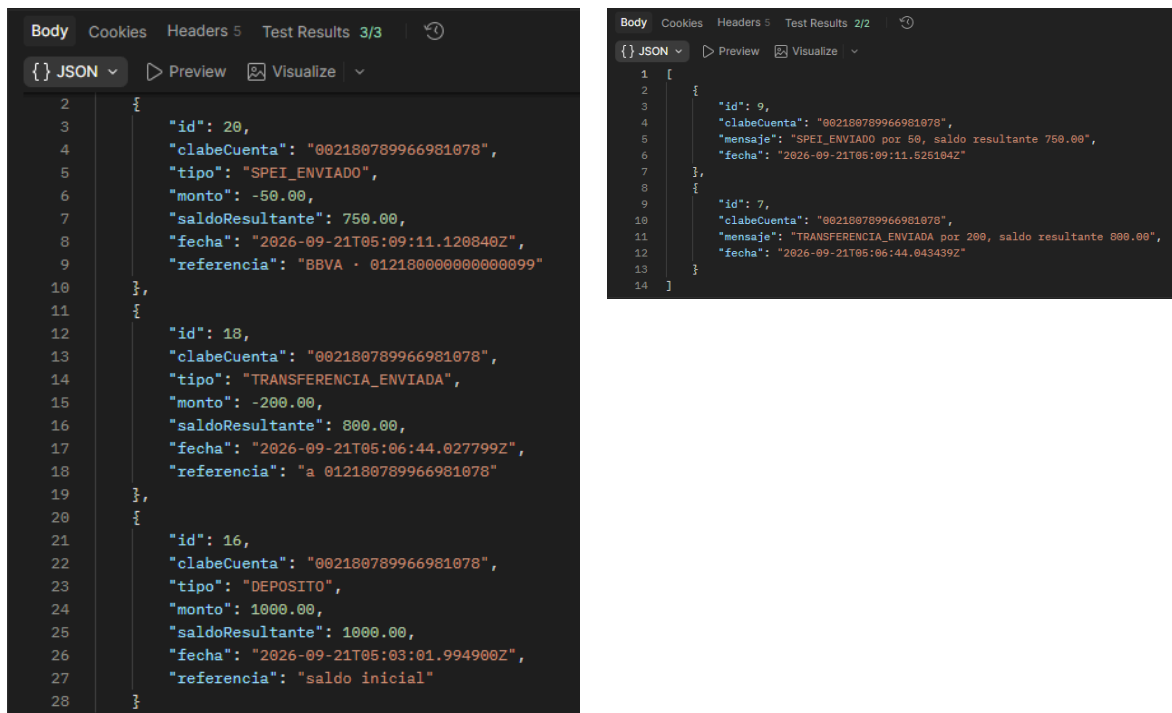
```
1 {
2   "id": 9,
3   "idempotencyKey": "prueba-789966981078",
4   "claveOrigen": "002180789966981078",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50.00,
8   "estado": "ENVIADO"
9 }
```



```
1 {
2   "id": 10,
3   "clabe": "002180789966981078",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

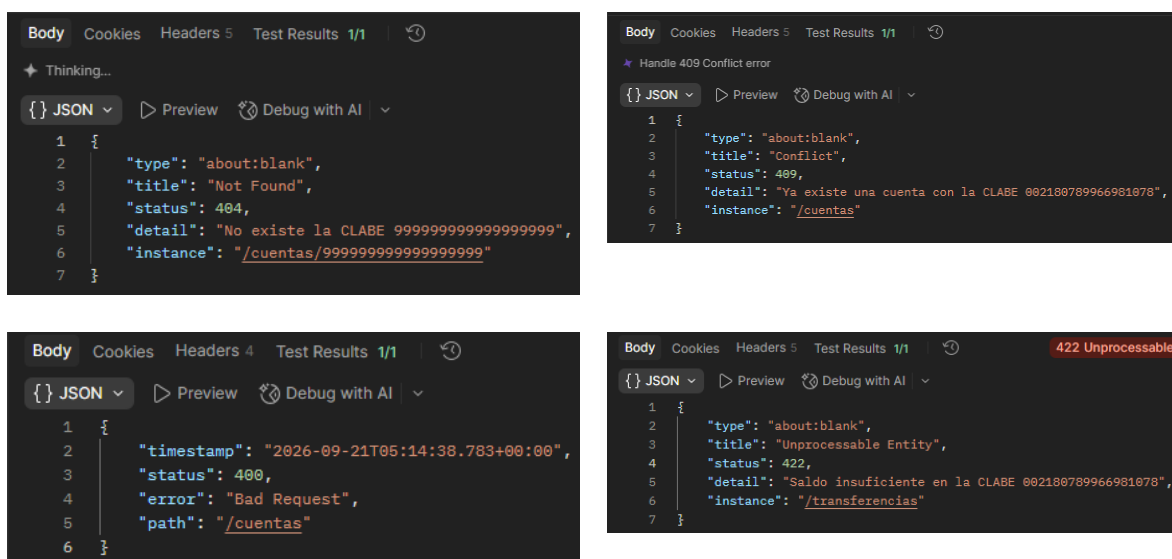
El estado de cuenta de Ana presentó tres movimientos: el depósito inicial de \$1000, la transferencia de -\$200 y un solo SPEI de -\$50, con saldo resultante \$750. Los avisos son dos: uno por la transferencia y otro por el SPEI.

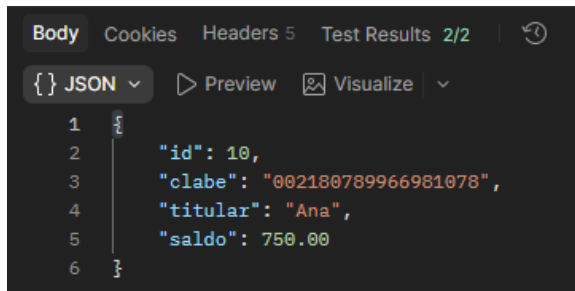
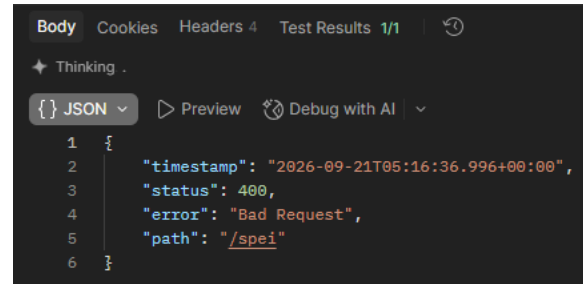
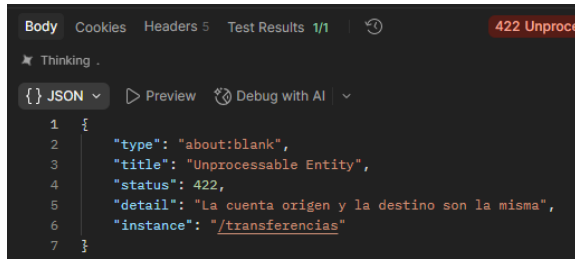
Imagen 6. El monolito por dentro.



Finalmente se probaron los casos de error de la colección: 404 Not Found al consultar una CLABE que no existe, 409 Conflict al intentar registrar una cuenta con una CLABE ya existente, 422 Unprocessable Entity al tratar de transferir más del saldo disponible o a la misma cuenta y 400 Bad Request al mandar una petición con algún campo faltante. En ninguno de los casos se modificó el saldo de la cuenta

Imagen 7. Cuando algo sale mal.





Paso 4. Bajar con y sin volumen.

Con docker compose down se eliminaron los contenedores y la red, pero no el volumen. Al volver a levantar, la cuenta de Ana seguía con un saldo de \$750.

Imagen 8. Down.

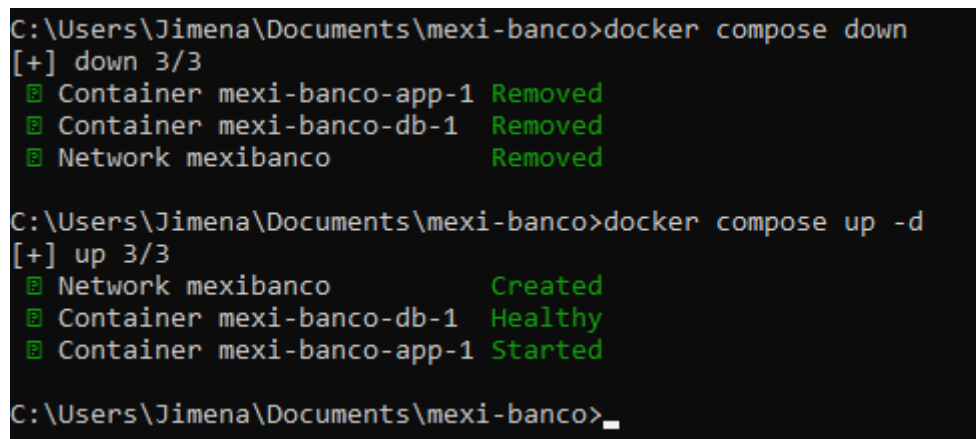
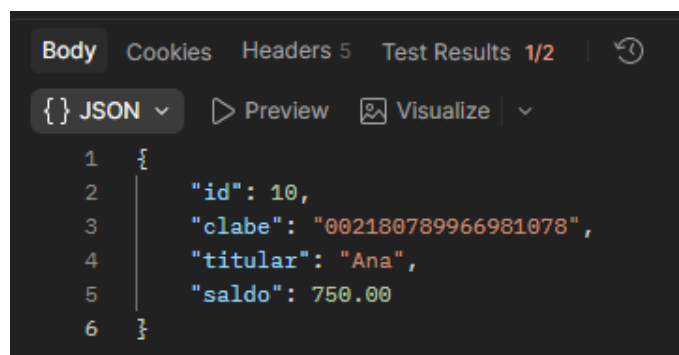


Imagen 9. Consulta después de compose down



Con Docker compose down -v también se eliminó el volumen y la misma consulta respondió con 404.

Imagen 10. Down -v.

```
C:\Users\Jimena\Documents\mexi-banco>docker compose down -v
[+] down 4/4
  Container mexi-banco-app-1      Removed
  Container mexi-banco-db-1      Removed
  Network mexibanco              Removed
  Volume mexi-banco_mexibanco_datos Removed

C:\Users\Jimena\Documents\mexi-banco>docker compose up -d
[+] up 4/4
  Network mexibanco              Created
  Volume mexi-banco_mexibanco_datos Created
  Container mexi-banco-db-1      Healthy
  Container mexi-banco-app-1     Started
```

Imagen 11. Consulta después de compose down -v

```
Body Cookies Headers 5 Test Results 0/2
{} JSON Preview Debug with AI
1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 002180789966981078",
6   "instance": "/cuentas/002180789966981078"
7 }
```

Los datos viven en el volumen y no en el contenedor. Los contenedores se pueden destruir y recrear sin problema; el volumen es lo que hay que cuidar, porque ahí es donde PostgreSQL guarda sus archivos: solo se pierden si el volumen se borra.

Paso 5. Revisión del código.

El código está organizado en cinco módulos: cuenta, movimiento, transferencia, notificación y spei. Cada uno está conformado por cuatro capas: controlador (traduce HTTP), servicio (reglas de negocio y transacciones), repositorio (acceso a su tabla) y entidad (la fila de la tabla). Cada módulo solo toca su propio repositorio y le pide lo demás al servicio del otro módulo.

En el SPEI, la idempotencia funciona insertando la solicitud con su Idempotency-Key en una columna UNIQUE: si el insert choca, se devuelve la solicitud ya guardada sin cobrar de nuevo. Así se evita que un reintento por una falla de red cobre dos veces.

Paso 6. Compose y Dockerfile.

docker-compose.yml. Define dos servicios, una red (mexibanco) y un volumen (mexibanco_datos). db usa postgres:17-alpine, guarda sus datos en el volumen y tiene un healthcheck con pg_isready. app se construye con el Dockerfile, publica el 8080, encuentra la base por el nombre del servicio (DB_HOST: db, sin IPs) y solo arranca cuando la base está healthy (depends_on con service_healthy); su healthcheck consulta /actuator/health. La configuración llega por variables de entorno; las credenciales están visibles a propósito porque son solo de desarrollo local (en un despliegue real irían en un secreto).

Dockerfile. Consta de dos etapas. La de construcción usa JDK 21, copia el pom.xml y descarga las dependencias antes de copiar el código y compila el .jar. La de ejecución usa solo JRE 21, copia el .jar y lo ejecuta. Así la imagen final es más pequeña, sin Maven, JDK ni código fuente.